

프로젝트 실행

ubuntu 20

```
# ~
sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get install git -y
sudo apt-get install curl -y
# build-essential
sudo apt update
sudo apt install build-essential
#vscode 설치하기
go, docker extension 설치
```

```
# docker
sudo apt-get install docker-compose -y
docker --version
docker-compose --version
sudo systemctl start docker
sudo systemctl enable docker

docker run hello-world
// permission denied? → sudo usermod -aG docker ${USER} → sudo reboot
docker run hello-world
```

```
# 기초 docker 명령어
동작중인 컨테이너 포함 모든 컨테이너 목록 확인
docker ps -a
도커 컨테이너 삭제
docker rm [container-id]
현재 설치된 도커 이미지 확인
docker images
도커 이미지 삭제
docker rmi [image-id]
```

```
#~
sudo apt-get update
sudo apt-get upgrade -y
```

```
# node.js
# https://github.com/nodesource
# nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.3/install.sh | bash
export NVM_DIR="$([ -z "${XDG_CONFIG_HOME-}" ] && printf %s "${HOME}/.nvm" || printf %s "${XDG_CONFIG_HOME-}" )"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
nvm -v
sudo reboot
# node.js, npm
nvm install l6
node -v
npm -v
```

```
# go
sudo apt remove 'golang-*'
wget https://go.dev/dl/go1.20.5.linux-amd64.tar.gz
tar xf go1.20.5.linux-amd64.tar.gz
sudo mv go /usr/local/go
gedit .profile
# 끝에서 아래 세줄 추가 후 ctrl x로 저장
export GOROOT=/usr/local/go
export GOPATH=$HOME/.go
export PATH=$PATH:$GOROOT/bin:$GOPATH/bin:$GOPATH/src/fabir-samples/bin

source .profile
go version
```

```
# jq
sudo apt-get install jq -y
```

```
# fabric-samples
echo $HOME
mkdir -p $HOME/.go/src
cd $HOME/.go/src/
curl -sSL https://bit.ly/2ysbOFE | bash -s
docker pull hyperledger/fabric-couchdb:latest
```

```
# 컨트랙트에서 go.mod, go.sum auto생성하기
cd {chaincode/path}
go mod init
go mod tidy
GO111MODULE=on go mod vendor
```

```
# 테스트네트워크 구동
cd fabric-samples/test-network
./network.sh down
./network.sh up
docker ps -a
./network.sh down
```

[1] 개발환경

1. vmware, ubuntu 18.04
2. cURL

URL을 통해 데이터를 전송할 수 있는 도구

```
$ sudo apt install curl
```

```
$ curl -V
```

1. 도커

도커는 애플리케이션을 컨테이너라는 계층으로 격리시켜 OS에 관계없이 컨테이너 안에서 개발, 배포, 운영할 수 있도록 제공하고, 어느 환경에서도 동일하게 실행

도커는 기본적으로 root권한이 필요, 리눅스 시스템의 현재 사용자를 도커 그룹에 추가해 관리자 이외의 권한으로도 도커 서버/클라이언트를 수행할 수 있도록 변경, 사용자 계정이 root 권한이 있다면 생략가능

```
$ sudo usermod -aG docker $USER
```

```
$ sudo reboot
```

```
$ curl -fsSL https://get.docker.com/ | sudo sh
```

1. 도커 컴포즈

<https://docs.docker.com/compose/install/#install-compose>, 도커 컴포즈의 각 OS별 설치 가이드를 확인가능

```
$ sudo curl -L "https://github.com/docker/compose/releases/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

도커 컴포즈 라이브러리에게 실행 권한을 주고, 버전을 확인해 설치가 잘 완료됐는지 확인

```
$ sudo chmod +x /usr/local/bin/docker-compose
```

```
$ docker-compose --version
```

1. Go

<https://golang.org/dl/>, go 언어의 최신 버전을 확인할 수 있다.

go언어를 설치할 /usr/local 디렉토리로 이동한 후 wget, tar 커맨드 명령어로 간단하게 설치

```
$ cd /usr/local
```

```
$ sudo wget https://storage.googleapis.com/golang/go1.11.1.linux-amd64.tar.gz
```

```
$ sudo tar -C /usr/local -xzf go1.11.1.linux-amd64.tar.gz
```

환경변수 설정, GOPATH는 외부 라이브러리나 패키지, 툴, 소스 등을 받아오는 위치를 지정

```
$ echo 'export PATH=$PATH:/usr/local/go/bin' | sudo tee -a /etc/profile && echo 'export GOPATH=$HOME/go' | tee -a $HOME/.bashrc && echo 'export PATH=$PATH:$GOROOT/bin:$GOPATH/bin' | tee -a $HOME/.bashrc && mkdir -p $HOME/go/{src,pkg,bin}
```

```
$ sudo reboot
```

1. Git

```
$ sudo apt update
```

```
$ sudo apt install -y git
```

```
$ git --version
```

1. python

```
$ sudo apt install -y python
```

```
$ python --version
```

1. node.js, npm

hlf에서는 Node.js를 하이퍼레저 컴포저, Node.js HyperledgerFabric SDK에서 사용한다. Node.js는 확장성 있는 네트워크 애플리케이션 개발에 사용되는 소프트웨어 플랫폼이다. 작성언어로 자바스크립트를 사용, 자체적으로 HTTP 서버 라이브러리를 포함하고 있기 때문에, 아파치와 같은 별도의 웹 서버가 없어도 웹 서비스를 제공할 수 있다. Node.js는 커뮤니티 개발을 위주로 자주 업그레이드가 된다. 개발중인 관련 모듈이 업그레이드를 따라가게 하기 위해서 버전의 변경을 자유롭게 하는 NVM환경에서 개발한다.

```
$ curl -o- https://raw.githubusercontent.com/creationix/nvm/v0.33.2/install.sh | bash
```

```
$ sudo reboot
```

```
$ nvm install 8
```

nvm use 커맨드 명령어로 Node.js 버전을 지정, Node.js 8 버전을 사용하기 위해서 다음의 명령어를 사용

```
$ nvm use 8
```

npm은 자바스크립트 프로그래밍 언어를 위한 패키지 관리자, Node.js를 설치하면 npm이 설치된다. 5.6.0버전 설치

```
$ npm install npm@5.6.0 -g
```

```
$ node -v
```

```
$ npm -v
```

1. vscode

사이트가서 설치

```
$ code
```

1. 자바 JDK

hlf에서 자바로 체인코드를 개발하기 위해서 자바를 설치, PPA를 추가, apt 업데이트를 진행, 자바 버전 8 설치

```
$ sudo add-apt-repository ppa:openjdk-r/ppa
```

```
$ sudo apt update
```

```
$ sudo apt install openjdk-8-jdk openjdk-8-jre
```

1. Gradle

자바에서 Build, Deploy에 자유도를 주면서, repository와 Dependency를 같이 취함

```
$ sudo apt install gradle
```

```
$ gradle -v
```

1. IntelliJ IDEA

사이트가서 다운받고 압축풀고 bin폴더의 idae.sh 실행

1. hyperledger fabric

하이퍼레저 패브릭을 설치하고자 하는 디렉토리로 이동해 설치를 시작

특정버전을 설치하고 싶은경우 `curl -sSL https://bit.ly/2ysbOFE | bash -s -- <fabric_version> <fabric-ca_version>`

```
$ cd $GOPATH/src
```

```
$ curl -sSL https://bit.ly/2ysbOFE | bash -s -- 1.4.12 1.4.9 0.4.22
```

[2] network

1. 예제를 통해 블록체인 네트워크를 생성하고 시작, 종료

```
$ cd $GOPATH/src/fabric-samples/first-network
```

1. 옵션에 대한 도움말 확인

```
$ ./byfn.sh -h
```

1. 네트워크 구성 및 환경 설정 파일을 생성, cryptogen 도구를 사용해 인증서를 생성 > configtxgen 도구를 사용해 오더링 서비스 노드의 제네시스 블록을 생성 > configtxgen 도구를 사용해 채널을 생성 > configtxgen 도구를 사용해 Org1MSP에 대한 앵커 피어 노드 트랜잭션 파일을 생성

```
$ ./byfn.sh generate
```

1. 네트워크에 관련된 도커 컨테이너를 생성하고 실행

```
$ ./byfn.sh up
```

1. 도커 컨테이너로 실행했던 네트워크를 종료하고 관련 파일을 삭제해 설정을 초기화

```
$ ./byfn.sh down
```

- . 파일 설명

configx.yaml, 네트워크의 채널과 제네시스 블록을 만들고 앵커 피어 노드 파일을 생성하기 위한 설정 파일이다.

crypto-config.yaml, MSP에서 인증서를 발급하기 위한 설정파일, cryptogen 툴로 인증서를 발급할 때 사용되는데, 이 파일을 통해 조직과 참가자들에게 각각의 인증서를 발급

docker-compose-cli.yaml, ./base/docker-compose-base.yaml, ./base/peer-base.yaml, 전체 네트워크 노드의 도커 컴포즈 설정 파일

- . BYFN 네트워크 공부전 셋팅

우선 fabric-samples의 first network 디렉토리를 복사

```
$ cd $GOPATH/src/fabric-samples/
```

```
$ cp -r ./first-network ./first-network-copy
```

3-1. cryptogen 도구를 사용해 인증서를 생성

```
$ cd $GOPATH/src/fabric-samples/first-network-copy
```

```
$ code crypto-config.yaml
```

BYFN에 관련된 인증서를 생성해 crypto-config 디렉토리에 생성, 확인해보자

```
$ ../bin/cryptogen generate --config=./crypto-config.yaml
```

```
$ ls crypto-config
```

3-2. configtxgen 도구를 사용해 오더링 서비스 노드의 제네시스 블록을 생성

configtxgen 도구를 사용해야 오더링 서비스 노드에 대한 제네시스 블록을 생성할 수 있다. configtxgen 도구의 설정 파일은 configtx.yaml 파일로 제공되며 제네시스 블록의 설정과 관련된 부분은 profiles에 명시돼 있다.

configtx.yaml 파일 안의 'profile'에서 TwoOrgsOrdererGenesis를 사용, Capabilities, Order, Consortiums로 구성된다. 다음은 위오프파일 및 profiles를 가지고 -outputBlock 옵션을 사용해 제네시스 블록을 생성하는 명령어이다.

```
$ export FABRIC_CFG_PATH=$PWD
```

```
$ ../bin/configtxgen -profile TwoOrgsOrdererGenesis -outputBlock ./channel-artifacts/genesis.block
```

3-3. configtxgen 도구를 사용해 채널을 생성한다

345는 configtx.yaml파일의 profiles 섹션에서 TwoOrgsChannel을 사용한다. TwoOrgsChannel의 프로파일을 사용하며 Consortium, Application으로 구성된다.

채널 이름은 자주 사용되기 때문에 export CHANNEL_NAME = mychannel을 이용해 환경변수로 지정한다. -

outputCreateChannelTx 옵션을 사용해 트랜잭션 파일을 생성한다. 생성되는 채널에 대한 이름은 channelId에 앞에서 변수로 지정한 \$CHANNEL_NAME을 이용해 mychannel로 지정했다.

```
$ export CHANNEL_NAME=mychannel
```

```
$ ../bin/configtxgen -profile TwoOrgsChannel -outputCreateChannelTx ./channel-artifacts/channel.tx -channelID $CHANNEL_NAME
```

3-4.configtxgen 도구를 사용해 Org1MSP에 대한 앵커 피어 노드 트랜잭션 파일을 생성한다.

3에서 사용했던 프로파일 옵션으로 TwoOrgsChannel을 그대로 사용하고 앵커 피어 노드를 생성하는 트랜잭션 파일 이름으로 Org1MSPanchors.tx를 적어준다. 다음으로 -outputAnchorpeersUpdate 옵션을 사용해 저장될 앵커 피어 노드 지정 트랜잭션 파일 위치를 만들어 줬다. 채널을 환경변수로 지정해줬던 CHANNEL_NAME을 사용한다. 다음은 Org1 조직에서 앵커 피어 노드에 대한 트랜잭션을 생성하는 전체 커맨드 명령어다.

```
$ ../bin/configtxgen -profile TwoOrgsChannel -outputAnchorPeersUpdate ./channel-artifacts/Org1MSPanchors.tx -channelID $CHANNEL_NAME -asOrg Org1MSP
```

3-5. configtxgen 도구를 사용해 Org2MSP에 대한 앵커 피어 노드 트랜잭션 파일을 생성한다.

```
$ ../bin/configtxgen -profile TwoOrgsChannel -outputAnchorPeersUpdate ./channel-artifacts/Org2MSPanchors.tx -channelID $CHANNEL_NAME -asOrg Org2MSP
```

- . BYFN 네트워크 관련 컨테이너가 도커 컨테이너에 제대로 올라갔는지 확인

실행된 컨테이너의 개수는 총 9개이며 테이블 형태로 보이는 것을 확인, 5개의 노드 컨테이너를 제외하고 cli와 3개의 체인코드가 컨테이너에 추가된것을 확인

한개의 cli 컨테이너는 BYFN 외부에서 각종 컨트롤이나 실행을 위한 사용자 개발 환경 컨테이너이며, 3개의 체인코드 컨테이너는 Peer 도구로 설치할 때 만들어지는 각 피어 노드 컨테이너에 연결돼 실행되는 컨테이너이다

```
$ docker ps
```

4-1. BYFN네트워크의 프로세스

(1) 5개의 노드(오더링, 4개의 피어노드) 컨테이너, CLI컨테이너가 실행되며 총 6개의 컨테이너가 먼저 실행

(2) 네트워크 내부의 CLI 컨테이너에 접속해 생성된 채널 트랜잭션 파일인 channel.tx를 가지고 채널을 생성하고 채널 mychannel을 생성하고 모든 피어 노드(2개의 조직, 4개의 피어 노드)를 가입시키킨다.

(3) 두조직의 peer0을 앵커로 가입한다.

(4) 피어 노드에 체인코드를 설치하고 확인하는 단계 이 단계에서 해당 피어 노드의 체인코드를 담당하는 체인코드 컨테이너 3개가 실행된다.

두 조직의 앵커 피어 노드(peer0.org1, peer0.org2)에 체인코드를 설치한다.

peer0.org2에 초깃값 a에 100을, b에 200을 설정한다.

peer0.org1에서 a의 값을 조회한다.

트랜잭션 호출로 a에서 b로 10만큼 이동한다.

체인코드를 갖고 있지 않은 peer1.org2에 체인코드를 설치한다.

peer1.org2에서 값 a를 조회한다. 초깃값인 100에서 전송한 10을 뺀 90이 조회한다.

4-2. 5개의 노드(오더링 서비스 노드, 4개의 피어노드), CLI 컨테이너, 총6개의 컨테이너를 실행

총 5개의 노드인 order.example.com, peer0.org1.example.com, peer1.org1.example.com, peer0.org2.example.com, peer1.org2.example.com는 hlf 네트워크에 배포될 때, 도커 이미지를 통해 도커 컨테이너로 구축된다.

이 작업을 수행하기 위해 \$GOPATH/src/fabric-samples/first-network-copy 디렉토리로 이동한다. 그리고 crypto-config.yaml 파일을 열어 내용을 확인해보면 총 6개의 컨테이너가 존재하는 것을 알 수 있다.

```
$ cd $GOPATH/src/fabric-samples/first-network-copy
```

```
$ code docker-compose-cli.yaml
```

이 컨테이너는 하이퍼레저 패브릭 네트워크를 구축하고 컨테이너 내부에서 명령을 실행하기 위해 필요하다. 위의 구성대로 도커 컴포즈로 다음과 같이 컨테이너를 생성하고 실행한다.

```
$ docker-compose -f docker-compose-cli.yaml up -d
```

docker ps 명령을 실행해 실행중인 컨테이너를 확인하면 앞서 했던 과정의 6개의 컨테이너가 실행중임을 알 수 있다.

```
$ docker ps -a
```

4-3. CLI컨테이너에 접속해 생성된 채널 트랜잭션 파일인 channel.tx를 가지고 채널 mychannel을 생성하고 모든 피어를 가입시킨다.

(1) BYFN의 모든 구성 요소는 CLI컨테이너 내에서 커맨드 명령어를 사용해 트랜잭션으로 실행된다. localhost에서 CLI컨테이너의 쉘에 액세스하기 위해 다음의 커맨드 명령어를 입력

```
$ docker start cli
```

```
$ docker exec -it cli bash
```

cli 컨테이너 안으로 접근했다. CLI컨테이너에서 각피어 노드에게 명령어를 보내는 경우 4개의 환경변수

(CORE_PEER_MSPCONFIGPATH, CORE_PEER_ADDRESS, CORE_PEER_LOCALMSPID, CORE_PEER_TLS_PROTECT_FILE)을 각각 변경해줘야한다.

(2) peer0.org1에 명령어를 보내는 경우 다음과 같이 변경한다.

```
#
```

```
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.ex
```

```
# CORE_PEER_ADDRESS=peer0.org1.example.com:7051
```

```
# CORE_PEER_LOCALMSPID="Org1MSP"
```

```
#
```

```
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1
```

채널 이름인 mychannel을 컨테이너 내부 환경에서도 적용한다. 다음으로 peer channel create 커맨드 명령어를 사용해 채널을 생성한다.

```
# export CHANNEL_NAME=mychannel
```

```
# peer channel create -o orderer.example.com:7050 -c $CHANNEL_NAME -f ./channel-artifacts/channel.tx --tls --cafile
```

```
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.example.com/tls/ca.crt
```

이후 peer channel create 커맨드 명령어를 사용해 채널이 제대로 생성되면 mychannel.block 파일 생성될 것이다.

```
# ls
```

채널이 생성됐으면 이제 피어를 채널에 참여시킬 수 있는데, 채널에 피어를 추가하기 위해서 CLI에서 peer channel join 명령어를 변수 변경하지 않고 수행하면 Org1의 peer0에 적용되어 mychannel에 가입시킨다.

```
# peer channel join -b mychannel.block
```

(3) peer1.org1을 채널에 가입시킨다.

```
#
```

```
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.ex
```

```
# CORE_PEER_ADDRESS=peer1.org1.example.com:8051
```

```
# CORE_PEER_LOCALMSPID="Org1MSP"
```

```
#
```

```
CORE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1
```

```
# peer channel join -b mychannel.block
```

(4) peer0.org2를 채널에 가입시킨다.

```

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.ex
# CODE_PEER_ADDRESS=peer0.org2.example.com:9051
# CODE_PEER_LOCALMSPID="Org2MSP"

#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2
# peer channel join -b mychannel.block
(5) peer1.org2를 채널에 가입시킨다.

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.ex
# CODE_PEER_ADDRESS=peer0.org2.example.com:10051
# CODE_PEER_LOCALMSPID="Org2MSP"

#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2
# peer channel join -b mychannel.block
4-4. 두 조직의 peer0를 앵커로 가입한다.
org1, org2의 peer0를 앵커 피어 노드로 가입하는 방법으로 채널을 업데이트한다.
(1)peer0.org1을 앵커 피어 노드로 업데이트한다.

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.ex
# CODE_PEER_ADDRESS=peer0.org2.example.com:7051
# CODE_PEER_LOCALMSPID="Org1MSP"

#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1
# peer channel update -o orderer.example.com:7050 -c $CHANNEL_NAME -f ./channel-artifacts/Org1MSPanchors.tx --
tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.examp
cert.pem
(2)peer0.org2를 앵커 피어 노드로 업데이트한다.

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.ex
# CODE_PEER_ADDRESS=peer0.org2.example.com:9051
# CODE_PEER_LOCALMSPID="Org2MSP"

#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2
# peer channel update -o orderer.example.com:7050 -c $CHANNEL_NAME -f ./channel-artifacts/Org1MSPanchors.tx --
tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.examp
cert.pem
4-5. 피어 노드에 체인코드를 설치하고 확인하는 단계
총3개(peer0.org1, peer0.org2, peer1.org2)의 체인코드 컨테이너를 추가한다.
(1)두 조직의 앵커 피어 노드(peer0.org1, peer0.org2)에 체인코드를 설치한다.

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.ex
# CODE_PEER_ADDRESS=peer0.org1.example.com:7051
# CODE_PEER_LOCALMSPID="Org1MSP"

```

```

#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1
# peer chaincode install -n mycc -v 1.0 -p github.com/chaincode/chaincode_example02/go/
#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.ex
# CODE_PEER_ADDRESS=peer0.org2.example.com:9051
# CODE_PEER_LOCALMSPID="Org2MSP"
#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2
# peer chaincode install -n mycc -v 1.0 -p github.com/chaincode/chaincode_example02/go/
(2)채널에서 체인코드를 인스턴스화하고 peer0.org2의 초기값에 a에 100, b에 200을 설정해서 채널의 체인코드가 초기화되고, 체인코
드 승인 정책이 실행되며 대상 피어에 대한 체인 노드 컨테이너가 실행된다.

# peer chaincode instantiate -o orderer.example.com:7050 --tls --cafile
/opt/gopathsrc/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.examp
cert.pem -C $CHANNEL_NAME -n mycc -v 1.0 -c '{"Args":["init","a","100","b","200"]}' -P "OR('Org1MSP.peer',
'Org2MSP.peer')"
(3) query 명령어를 이용해 체인코드를 확인해보자. peer0.org1에서 a의 값을 조회한다.

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1.ex:
# CODE_PEER_ADDRESS=peer0.org2.example.com:7051
# CODE_PEER_LOCALMSPID="Org1MSP"
#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org1
# peer chaincode query -C $CHANNEL_NAME -n mycc -c '{"Args":["query","a"]}'
(4) 트랜잭션 호출로 a에서 b로 10만큼 이동한다.

# peer chaincode invoke -o orderer.example.com:7050 --tls --cafile
/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/ordererOrganizations/example.com/orderers/orderer.examp
cert.pem -C $CHANNEL_NAME -n mycc -c '{"Args":["invoke","a","b","10"]}'
(5) 체인코드를 갖고 있지 않은 peer1.org2에 체인코드를 설치한다.

#
CORE_PEER_MSPCONFIGPATH=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2.ex
# CODE_PEER_ADDRESS=peer0.org2.example.com:10051
# CODE_PEER_LOCALMSPID="Org2MSP"
#
CODE_PEER_TLS_ROOTCERT_FILE=/opt/gopath/src/github.com/hyperledger/fabric/peer/crypto/peerOrganizations/org2
# peer chaincode install -n mycc -v 1.0 -p github.com/chaincode/chaincode_example02/go/
(6) peer1.org2에서 값 a를 조회 시에 초기값인 100에서 전송한 10을 뺀 90이 조회된다.

# peer chaincode query -C $CHANNEL_NAME -n mycc -c '{"Args":["query","a"]}'
5.작업 구축한 네트워크를 삭제하자

# exit
$ docker-compose -f docker-compose-cli.yaml down
$ rm -rf channel-artifacts/*.block channel-artifacts/*.tx crypto-config
$ docker rm $(docker ps -aq) -f
$ docker system prune --volumes -f

```